

UST-2026-09-26-19

Alta pública blindada: la lista de espera se apunta por una puerta (Edge waitlist-join tras /api/waitlist en nuestro dominio) con honeypot, ritmo por IP, Origin y Turnstile; el INSERT anónimo por REST se retira; el digest avisa del crecimiento

PROPUESTA · alcance validado por Juanjo en el chat (26-sep: «2+3+4+5+7» de las siete capas propuestas) · pendiente de validación fila a fila (D1-D10). Nada construido. Lo que se afirma está leído del código (repos/web2/src/js/30-forms.js, tools/patches/waitlist.py, integrate.py ns-waitlist-js, Edge community-lead) o medido en la base el 26-sep.

1 · El pedido, literal

Juanjo, 26-sep 2026, ≈23:00, tras cerrar el incidente de la waitlist (UST-18):

«como se puede prevenir que un bot meta esos correos falsos»

«2+3+4+5+7» (sobre las siete capas propuestas: 2 cerrar REST y pasar por Edge · 3 honeypot, ritmo por IP, Origin, tiempo mínimo · 4 proxy en nuestro dominio con WAF/rate limiting de Cloudflare · 5 Turnstile · 7 vigilancia diaria). Fuera del alcance: 6 (doble opt-in → ☐ WL-2, carril producto).

Lectura: que apuntarse a la lista de espera sea un CAMINO ÚNICO y vigilado (nuestro dominio → Cloudflare → Pages Function → Edge → base), y que el camino viejo (POST directo a PostgREST con la anon key) deje de existir. Un bot no se «disuade»: se le quita la puerta.

Principio (stack lean, Juanjo 2-ago)

Sin proveedores nuevos: todo con Cloudflare (Pages Functions, WAF, Turnstile) y Supabase (Edge, SQL) que ya tenemos. Cero servidores propios. Coste: 0 €.

2 · Lo que hay hoy (leído del código y medido en la base)

Qué	Medido
Cómo se apunta alguien	web2 (`src/js/30-forms.js`): `fetch(SB + "/rest/v1/waitlist")` con la anon key y `{email, source:"web2"}`; sin JS, el ` <form>` (`tools/patches/waitlist.py`) hace POST directo a `https://nebkqncvapelrarruyqb.supabase.co/rest/v1/waitlist`. El marketing viejo en `/v1/` (`integrate.py` bloque `ns-waitlist-js`) hace lo mismo con `marketing_site`; el pie, `newsletter`. La cifra de la Home sale de la RPC `waitlist_count` (anon).</form>
La anon key	Pública por diseño (RLS manda) y visible en el HTML de la web: cualquiera puede llamar al endpoint sin pasar por el formulario. Eso hizo el bot: 630.076 POST en dos ventanas, sin abrir la web.
Guardas de hoy (UST-18)	Tope de lote (>20 filas/sentencia) · origen obligatorio para anon (web2 / marketing_site / newsletter) · ritmo GLOBAL 200 altas/10 min · CHECK de formato · `created_at` lo pone el servidor. Probadas en producción (curl → 400/23514).
Lo que las guardas NO ven	La IP (un trigger no la tiene de forma fiable), si hay un humano detrás, si la petición viene de nuestra web. Y el ritmo es global: 200/10 min frenaría también un pico legítimo.
Lo que Cloudflare ve hoy	Nada del alta: el POST va del navegador (o del bot) directo a `supabase.co`. WAF, rate limiting y bots de Cloudflare protegen nutrisynccollective.com, no el endpoint.
El patrón ya existe en casa	Edge `community-lead` (5/6-sep): honeypot `website`, ritmo por email (3/24 h) y por hash de IP (10/h) vía RPC `community_lead_allowed`, Origin en CORS, 303 → página de gracias sin JS, nunca guarda la IP (sha256 truncado). `waitlist-join` es su gemela.
Vigilancia	Ninguna: 630.083 filas estuvieron 16 días sin que nadie las viera. El digest de las 07:15 (`founder_pending_summary`) no mira la waitlist.

Analogía: hoy la base de datos tiene un puerto publicado a Internet con una ACL por fila (RLS). La arquitectura buena es la de cualquier API: un API gateway en nuestro dominio (Pages Function detrás del WAF de Cloudflare) que autentica la petición (Turnstile = «hay un humano»), la limita (rate limit por IP), la valida (honeypot, Origin, formato) y la reenvía al backend por un canal que solo él conoce (la Edge con un secreto compartido); y el puerto del backend deja de escuchar a Internet (se retira la policy de INSERT anónimo).

3 · Cambios, fila a fila (propuesta)

Qué	Qué se construirá	Cómo se verificará
C1 · Edge `waitlist-join`	Gemela de `community-lead`: acepta JSON (fetch) o form-urlencoded (sin JS); campos `email`, `source` (solo de la lista blanca: web2 / marketing_site / newsletter), `locale`, `website` (honeypot), `cf-turnstile-response`, `t0` (instante de carga del formulario). Rechaza si: honeypot relleno (responde OK y no guarda), Origin fuera de nuestros dominios, formulario enviado < 2 s tras cargarse, Turnstile inválido (cuando hay token), o `waitlist allowed(email, ip_hash)` dice no. Inserta con service_role (Vault). Exige la cabecera `x-ns-gateway` (secreto compartido con la Pages Function): sin ella, 403 — así nadie llama a la Edge saltándose Cloudflare. 409 si el correo ya está; 303 → gracias/error sin JS; JSON `{ok}` con JS.	Arnés de funciones puras (transpilar con el `typescript` de appdev, como community-lead) para: lista blanca de source, honeypot, tiempo mínimo, forma del hash de IP. Estreno REAL con una llamada por cada camino (r21): fetch OK · sin JS 303 · honeypot 200-sin-fila · sin gateway 403 · Turnstile inválido 400 · duplicado 409.
C2 · SQL: ritmo por IP y cierre del REST	`waitlist allowed(p_email, p_ip_hash)` security definer (≤ 3 altas/24 h por email · ≤ 5/h por hash de IP; sin JS/Turnstile, ≤ 3/h) sobre `waitlist_ritmo` (ip_hash, email_hash, created_at; purga > 48 h en la propia función). `drop policy` "anyone can join waitlist" + `revoke insert on waitlist from anon` (authenticated conserva el grant para la policy `admins * waitlist` del hub). Las guardas de UST-18 se quedan (defensa en profundidad; la Edge entra como service_role y no las pisa).	Fichero con `begin/commit`, autoprueba (`set local role anon` → INSERT = 42501 permission denied) y verificación final. Arnés PG16 antes de la bandeja. Se ejecuta EL ÚLTIMO (D10): cuando la web ya envía por la puerta nueva.
C3 · Pages Function `/api/waitlist` (webdeploy)	`repos/webdeploy/functions/api/waitlist.js`: recibe el POST (JSON o form), añade `cf-connecting-ip` como `x-ns-ip` y `x-ns-gateway` (secreto en variable de entorno del proyecto Pages), reenvía a la Edge y devuelve su respuesta (JSON o 303). Misma URL para web2 (raíz), `/v1/` y m. Ventaja: mismo origen (sin CORS) y la anon key sale del JS de la web.	web-tests: el módulo se ejecuta en vm con peticiones de forma (JSON, form, honeypot) contra una Edge falsa. smoke-prod: `POST /api/waitlist` con honeypot → 200 `{ok:true}` y `waitlist_count` NO cambia.
C4 · Cloudflare: WAF y rate limiting (Juanjo, panel)	Security → WAF → Rate limiting rules: `http.request.uri.path eq "/api/waitlist"` → 10 peticiones / 10 s por IP → bloquear 10 s (la regla gratuita del plan). Bot Fight Mode: NO (D6) — desafiaría también a smoke-prod, curl y Lighthouse, y en el plan Free no admite excepciones.	Prueba con `ab`/bucle de curl desde el Mac: la 11ª petición en 10 s devuelve 429 de Cloudflare; la Edge no la ve.
C5 · Turnstile (Cloudflare) en los formularios	Widget «Managed» (invisible salvo sospecha) en los 3 formularios de alta (web2 Home + Community, `/v1/`, pie newsletter). Site key en el HTML (pública); secret en `supabase secrets` para la Edge, que verifica en `challenges.cloudflare.com/turnstile/v0/siteverify` con la IP. El script `https://challenges.cloudflare.com/turnstile/v0/api.js` es una DESVIACIÓN DECLARADA de po74 («cero CDNs de terceros»): mismo proveedor que ya sirve la web y ya ve todas las peticiones; lista blanca explícita, con el motivo, en pre-deploy (bloque 7), verify-web-pack y smoke. Pilar corrobora y, si procede, una línea en la política de privacidad.	Capturas ES/EN × 390/1440 (textfit no debe cambiar: el widget va debajo del botón). En vivo: alta con token válido → 201; sin token con JS → 400 «verificación pendiente»; sin JS → camino C1 con límite estricto.
C6 · web2 y `/v1/`: el formulario habla con `/api/waitlist`	`30-forms.js`: `fetch("/api/waitlist")` (mismo origen), cuerpo `{email, source, locale, website, t0, cf-turnstile-response}`; `tools/patches/waitlist.py`: `action="/api/waitlist"` (sin JS). `integrate.py` bloque `ns-waitlist-js` (marketing viejo en `/v1/`): misma URL. Mensajes ya existentes (ok/dup/err) + uno nuevo «verificación pendiente».	verify-web2 + textfit en 5 idiomas × 2 anchos · candidata `v2.` verificada por Juanjo (circuito de 5 pasos, «sin 2 y 3 no hay 4 o 5») · web-tests (snippet `ns-waitlist-js` real en vm).
C7 · Vigilancia en el digest de las 07:15	`founder_pending_summary` añade `waitlist altas 24h` y `waitlist_total`; `founder-reminders` escribe una línea siempre («  lista de espera de 24h en la base de datos»). Además, una sonda de solo lectura re-ejecutable (`-sonda-waitlist-crecimiento.sql`).	Estreno con una llamada real a la Edge (mirar el cuerpo, `cf-turnstile-response`), y en la consola de la marca  altas en 24 h → aparece el  .
C8 · Orden de entrega (el formulario nunca se queda sin puerta)	(1) SQL de `waitlist_ritmo` + `waitlist_allowed` → (2) `supabase secrets` (Turnstile secret + gateway) y Deploy Edge con verificación en vivo → (3) variable `NS_GATEWAY` en Pages (nutrisync-collective) → (4) web2 candidata → verificada → integrate → Deploy Web (po) → (5) smoke → (6) SOLO ENTONCES el SQL que retira la policy y el grant → (7) smoke otra vez: POST anon a `/rest/v1/waitlist` = 401/42501.	Cada paso con su comprobación antes del siguiente (regla 7-sep: validar antes de sobrecribir). Baseline antes del paso 1.

4 · Análisis de impacto

Dónde	Qué toca	Qué NO toca
repos/web2	`src/js/30-forms.js`, `tools/patches/waitlist.py` (+ Turnstile en la barra), catálogo i18n (1 clave × 5 idiomas), `build.py` sin cambios de forma → circuito de 5 pasos completo.	Layout, imágenes, Community (sus formularios ya van por Edge), SEO, PWA shell.
repos/webdeploy	Nuevo `functions/api/waitlist.js`; `integrate.py` bloque `ns-waitlist.js` (URL) y lista blanca de Turnstile en pre-deploy/verify-web-pack; `_routes.json` si hace falta para `/api/*`.	Hub (□ Waitlist y □ Piloto siguen con `sb.from("waitlist")` como admins: policy `admins * waitlist` intacta), legal, adjuntos, middleware pages.dev.
Backend SQL	`waitlist_ritmo` (nueva), `waitlist allowed` (nueva), `founder_pending_summary` (v+1), drop policy + revoke insert anon. Las guardas de UST-18 se quedan.	Tabla `waitlist` (columnas y filas), RPC `waitlist_count` (sigue anon: lectura barata), `admin_waitlist` (paginación = item aparte S19).
Edge Functions	Nueva `waitlist-join`; `founder-reminders` (una línea nueva). Secrets: `TURNSTILE_SECRET`, `NS_GATEWAY_SECRET`.	`community-lead` (se copia, no se toca), `notify-dispatch`, crons.
Cloudflare (panel, Juanjo)	Turnstile: crear widget (site key + secret). WAF: 1 regla de rate limiting. Pages: variable de entorno `NS_GATEWAY`. Access: nada.	DNS, Zero Trust, Bot Fight Mode (NO se activa, D6).
Quién lo recibe	Quien se apunte desde la web (EN/ES/CA/GL/EU) y desde `/v1/`: mismo gesto, un widget más (normalmente invisible). Las founders: línea nueva en el digest.	Testers y usuarias de la app (la app no escribe en waitlist). Las 168 personas ya apuntadas.
Riesgos	Turnstile mal configurado = nadie se apunta → por eso el orden C8 y el camino sin token con límite estricto (D4) como red. Un fallo en la Edge = 303 a la página de error con motivo (nsFail, nada en silencio). La regla WAF demasiado agresiva = 429 a humanos rápidos (10/10 s por IP es holgado para una persona).	—

Desviación declarada (única): el script de Turnstile se carga de `challenges.cloudflare.com`. po74 exige cero CDNs de terceros; Cloudflare no es un tercero nuevo (sirve la web entera desde Pages y ya ve cada petición), pero es un script externo y se registra como excepción con motivo en las tres puertas. Pilar corrobora (compliance) y decide si la política de privacidad lo menciona.

5 · Decisiones (fila a fila — pendientes de validación; el alcance «2+3+4+5+7» ya está dado)

#	Propuesta	Alternativa	Decisión
D1	Cerrar el REST para anon: `drop policy "anyone can join waitlist" + `revoke insert ... from anon`. El POST directo a `/rest/v1/waitlist` pasa a 401/42501 para todo el mundo.	Dejar el REST abierto y añadir solo Turnstile en el HTML (no cierra nada: el bot no usa el HTML).	
D2	Puerta ÚNICA `/api/waitlist` en nuestro dominio (Pages Function) → Edge con `x-ns-gateway` (secreto compartido); la Edge rechaza (403) lo que no venga por la puerta. Así Cloudflare ve y limita TODO el tráfico de alta.	Edge pública como `community-lead` (llamable directo desde el navegador: Cloudflare no la ve y el WAF no aplica).	
D3	Turnstile modo «Managed» (invisible salvo sospecha), verificado en la Edge con la IP; site key pública en el HTML; desviación po74 declarada con lista blanca y motivo; Pilar corrobora.	Sin Turnstile (solo honeypot + ritmo). Más simple, pero un bot con navegador real y una IP distinta por petición sigue entrando al ritmo permitido.	
D4	Sin JS (o Turnstile caído): se ACEPTA el alta con honeypot + Origin + ritmo estricto (3/h por IP) — accesibilidad y red de seguridad si Turnstile falla.	Rechazar todo alta sin token de Turnstile.	

#	Propuesta	Alternativa	Decisión
D5	Ritmo por IP con hash sha256 truncado (nunca la IP en claro, como community-lead): 5/h por IP · 3/24 h por email; tabla `waitlist_ritmo` purgada a las 48 h dentro de la propia función.	Solo la regla de rate limiting de Cloudflare (por IP real, sin memoria entre peticiones lentas: 1 alta cada 11 s la pasa).	
D6	Regla WAF de rate limiting: `/api/waitlist`, 10 peticiones/10 s por IP → bloqueo 10 s. Bot Fight Mode NO (desafiaría a smoke-prod, curl, Lighthouse; en Free no admite excepciones).	Bot Fight Mode ON en toda la zona.	
D7	El tope global de UST-18 (200/10 min) se queda como red de seguridad de la base (defensa en profundidad), igual que el tope de lote y la guarda de origen.	Retirarlo al cerrar el REST.	
D8	Vigilancia: línea SIEMPRE en el digest de las 07:15 («+N en 24 h · total T») y Δ si N > 50; sonda re-ejecutable en backend-sql.	Cron propio + correo aparte (más piezas, mismo aviso).	
D9	Carril técnico: Juanjo decide y ejecuta; Pilar corrobora la desviación po74 (Turnstile) y el texto de privacidad si procede; Lucía, el widget bajo el botón (no cambia el diseño).	Carril producto (firma previa de las dos).	
D10	Orden C8: la web habla con la puerta nueva ANTES de cerrar la vieja; `drop policy` es el ÚLTIMO paso, con smoke antes y después. Entrega WEB en S19 (≈ 4-5 h de trabajo + los 5 pasos del circuito web2).	Cerrar primero el REST (deja el formulario roto entre pasos).	

6 · Baseline, verificación y marcha atrás

Momento	Qué se comprueba
PASO 0 · Baseline (antes de tocar)	`tools/baseline.sh` (sha de web2/webdeploy, sellos ns-build, candados) + lista de policies de `waitlist` (`pg_policies`) + `waitlist_count()` + `supabase functions list`. Sin baseline escrita no hay marcha atrás.
Test antes de desplegar	Arnés de puras de la Edge · arnés PG16 del SQL (incluida la autoprueba «anon INSERT = 42501») · web-tests (Pages Function + snippet ns-waitlist-js) · verify-web2 + textfit ×5 idiomas · pre-deploy bloque 7 con la lista blanca.
Estreno real (r21)	Un POST por camino contra producción: fetch OK (201, aparece en ☐) · sin JS (303 a gracias) · honeypot (200, no aparece) · sin gateway a la Edge (403) · Turnstile inválido (400) · duplicado (409) · 11 peticiones en 10 s (429 de Cloudflare) · POST anon a `/rest/v1/waitlist` (401/42501, tras D1).
DoD	Las tres verificaciones del párrafo anterior + una persona (Lucía) se apunta desde su móvil en ES y aparece en ☐ · el digest del día siguiente trae la línea de la waitlist.
Marcha atrás	SQL: `create policy "anyone can join waitlist" ... with check (true)` + `grant insert on waitlist to anon` (una sentencia cada una; fichero `rollback` preparado). Web: Cloudflare Pages → Rollback al deploy anterior (web2 candidata y webdeploy). Edge: no hace falta retirarla (sin la policy vieja la web seguiría funcionando por la puerta nueva). WAF: desactivar la regla.

7 · What to Test (para Pilar y Lucía, cuando esté desplegado)

1. Apuntarse desde la Home (ES y EN) con un correo propio: mensaje de «¡Estás dentro!» y la persona aparece en ☐ Waitlist en el acto. Normalmente no se ve ningún «¿eres humano?»; si aparece, es un clic.
2. Apuntarse otra vez con el mismo correo: «ya estabas en la lista» (409), sin duplicado en ☐.
3. Desde el pie (newsletter) y desde /v1/: lo mismo.
4. En el digest de las 07:15 del día siguiente: línea «☐ lista de espera: +N en 24 h · total T».
5. Para Juanjo: el curl del 26-sep (POST anónimo a /rest/v1/waitlist) devuelve ahora 401/42501 en vez de 400/23514 — la puerta ya no existe, no es que esté vigilada.

8 · La cadena

Paso	Estado
UST	Este documento (propuesta, 26-sep). Alcance «2+3+4+5+7» validado por Juanjo en chat; D1-D10 pendientes fila a fila.
Sign-off	Carril técnico: Juanjo valida D1-D10 (firma electrónica simple). Pilar corrobora la desviación po74 (D3) antes del Deploy Web. Doble opt-in queda fuera: □ WL-2 (Pilar y Lucía).
Dev	S19: SQL (ritmo + cierre REST) · Edge `waitlist-join` · Pages Function · web2 (formulario + Turnstile) · integrate.py (`ns-waitlist-js`, lista blanca) · founder-reminders · sonda.
Test	Arnés de puras · arnés PG16 · web-tests · verify-web2 + textfit · pre-deploy · estreno real por cada camino.
Deployment	Orden C8/D10: SQL ritmo → secrets + Deploy Edge → variable Pages → web2 candidata → verificada → integrate → Deploy Web → smoke → SQL cierre REST → smoke.
Verification	DoD arriba (técnica + Lucía desde su móvil + digest del día siguiente).
hub + Notion	Hub: este doc 54 + adjunto (po255); as-built al cerrar. Notion: sin bloques → hub.

Firmas

Persona	Rol	Conforme	Fecha
Juanjo Cebrián	Ingeniería y producto (D1-D10, carril técnico)		
Pilar González	CEO · cumplimiento (corrobora D3: Turnstile como excepción declarada a po74; privacidad)		
Lucía Cebrián	COO · diseño (widget bajo el botón; mensaje «verificación pendiente» en 5 idiomas)		